

Κεφάλαιο 6

Αρχεία και Επικοινωνία με Αρχεία στη Fortran

OPEN, READ, WRITE, CLOSE, IOSTAT, REWIND και δεδομένα αγνώστου πλήθους

Στόχος: να μπορείς να διαβάζεις δεδομένα από αρχείο, να γράφεις αποτελέσματα σε αρχείο και να χειρίζεσαι σωστά ένα αρχείο με άγνωστο αριθμό εγγραφών.

6.1 Γιατί χρησιμοποιούμε αρχεία

Μέχρι τώρα το πρόγραμμα διάβαζε δεδομένα από το πληκτρολόγιο και εμφάνιζε αποτελέσματα στην οθόνη. Αυτό είναι αρκετό για λίγες τιμές, αλλά όχι για μετρήσεις, πειραματικά δεδομένα ή αποτελέσματα που θέλουμε να κρατήσουμε. Ένα αρχείο επιτρέπει στο πρόγραμμα να ανταλλάσσει δεδομένα με τον δίσκο.

Παραδείγματα: ένα αρχείο μπορεί να περιέχει θερμοκρασίες πολλών ημερών, μετρήσεις από αισθητήρα ή τις τιμές μιας προσομοίωσης. Το πρόγραμμά μας μπορεί να τις διαβάσει, να υπολογίσει μέσους όρους και να γράψει μια καθαρή αναφορά σε νέο αρχείο.

Στο κεφάλαιο θα χρησιμοποιούμε αρχεία απλού κειμένου. Κάθε γραμμή είναι μία εγγραφή (record) και μέσα στη γραμμή υπάρχουν τιμές ή πεδία (fields), συνήθως χωρισμένα με κενά.

6.2 Η βασική διαδρομή: OPEN - επεξεργασία - CLOSE

Για να δουλέψει ένα πρόγραμμα με εξωτερικό αρχείο, ακολουθεί σχεδόν πάντα τρία βήματα: ανοίγει το αρχείο, κάνει READ ή WRITE και στο τέλος το κλείνει. Η Fortran χρησιμοποιεί έναν αριθμό μονάδας (unit) για να αναφέρεται στο ανοιχτό αρχείο.

```
open(unit=10, file='temperatures.txt', status='old')  
! READ ή WRITE με τη μονάδα 10  
close(10)
```

Το 10 δεν είναι μέγεθος, θέση ή πλήθος δεδομένων. Είναι απλώς ένας αριθμός-όνομα που συνδέει προσωρινά το πρόγραμμα με το αρχείο. Επιλέγουμε έναν θετικό αριθμό που δεν χρησιμοποιούμε αλλού στο ίδιο πρόγραμμα.

Στις επόμενες ενότητες θα εξηγήσουμε τα ορίσματα του OPEN και θα δούμε ολοκληρωμένα παραδείγματα.

6.3 Η εντολή OPEN

Η OPEN συνδέει μια μονάδα εισόδου/εξόδου με ένα αρχείο. Η μορφή που θα χρησιμοποιούμε πιο συχνά είναι:

```
open(unit=10, file='weatherdata.txt', status='old', action='read')
```

unit=10: η μονάδα που θα χρησιμοποιήσουμε αργότερα σε READ ή WRITE. file='weatherdata.txt': το όνομα του αρχείου. Αν γράψουμε μόνο όνομα, το αρχείο πρέπει να βρίσκεται στον ίδιο φάκελο με το πρόγραμμα ή με το project. status='old': το αρχείο πρέπει ήδη να υπάρχει. action='read': το ανοίγουμε μόνο για ανάγνωση.

Για να δημιουργήσουμε αρχείο αποτελεσμάτων, χρησιμοποιούμε συνήθως status='replace' και action='write'. Αν υπάρχει ήδη αρχείο με αυτό το όνομα, το replace το αντικαθιστά. Γι' αυτό επιλέγουμε προσεκτικά το όνομα του αρχείου εξόδου.

```
open(unit=20, file='results.txt', status='replace', action='write')
```

Χρήσιμη σύνοψη: old = υπάρχον αρχείο, συνήθως για δεδομένα εισόδου. replace = νέο αρχείο ή αντικατάσταση παλαιού, συνήθως για αποτελέσματα. action='read' επιτρέπει READ, action='write' επιτρέπει WRITE. Αν δεν ξέρουμε ακόμη τις επιλογές, η ασφαλής συνήθεια είναι να τις γράφουμε ρητά ώστε ο κώδικας να είναι ξεκάθαρος.

6.4 Διάβασμα μιας γνωστής ποσότητας δεδομένων

Αν γνωρίζουμε ότι το αρχείο έχει ακριβώς n γραμμές, ένας βρόχος DO αρκεί. Ας θεωρήσουμε ότι το temperatures.txt περιέχει μία πραγματική τιμή σε κάθε γραμμή.

```
program read_known
implicit none
integer, parameter :: n = 4
integer :: i
real :: t(n)

open(unit=10, file='temperatures.txt', status='old', action='read')
do i = 1, n
  read(10, *) t(i)
end do
close(10)

print *, t
end program read_known
```

Το read(10,*) t(i) διαβάζει από τη μονάδα 10 την επόμενη διαθέσιμη τιμή και την αποθηκεύει στη θέση i του πίνακα t. Το * είναι ελεύθερος μορφότυπος: οι τιμές μπορούν να χωρίζονται με ένα ή περισσότερα κενά.

6.5 Εγγραφή αποτελεσμάτων σε αρχείο

Η WRITE είναι γενική εντολή εξόδου: μπορεί να εμφανίσει κάτι στην οθόνη ή να το γράψει σε αρχείο. Ο προορισμός καθορίζεται από το πρώτο όρισμα, δηλαδή από τη μονάδα εξόδου που γράφουμε μέσα στην παρένθεση.

```
write(*, '(a,f8.2)') 'Μέση τιμή: ', average ! οθόνη  
write(20, '(a,f8.2)') 'Μέση τιμή: ', average ! αρχείο που συνδέθηκε με unit=20
```

Το * σημαίνει την προεπιλεγμένη έξοδο, δηλαδή την οθόνη. Επομένως write(*,...) εμφανίζει το αποτέλεσμα στο terminal. Το 20 σημαίνει «γράψε στο αρχείο που ανοίξαμε νωρίτερα με unit=20». Η PRINT είναι σύντομη μορφή για εμφάνιση στην οθόνη, π.χ. print '(a,f8.2)', 'Μέση τιμή: ', average λειτουργεί όπως write(*, '(a,f8.2)') 'Μέση τιμή: ', average.

```
program write_results  
implicit none  
real :: average  
  
average = 23.456  
open(unit=20, file='results.txt', status='replace', action='write')  
write(20, '(a,f8.2)') 'Μέση θερμοκρασία: ', average  
close(20)  
end program write_results
```

Η γραμμή write(20, '(a,f8.2)') γράφει στη μονάδα 20. Το a χρησιμοποιείται για το κείμενο 'Μέση θερμοκρασία: '. Το f8.2 γράφει τον πραγματικό αριθμό σε πεδίο πλάτους 8, με 2 ψηφία μετά την υποδιαστολή. Άρα στο results.txt θα γραφτεί μια καθαρή γραμμή με την τιμή 23.46.

Το CLOSE είναι απαραίτητο: ολοκληρώνει σωστά την επικοινωνία και εξασφαλίζει ότι τα δεδομένα που περιμένουν να γραφτούν θα περάσουν στο αρχείο. Μην αφήνεις ανοιχτή μονάδα όταν τελειώσεις με αυτήν.

6.6 Γιατί χρειαζόμαστε IOSTAT

Όταν δεν ξέρουμε πόσες γραμμές έχει ένα αρχείο, δεν μπορούμε να γράψουμε `do i=1,n`. Χρειαζόμαστε τρόπο να καταλάβουμε πότε η ανάγνωση έφτασε στο τέλος του αρχείου. Η επιλογή `iostat` αποθηκεύει σε ακέραιο μεταβλητή τον κωδικό κατάστασης της `READ`.

```
integer :: ierr
read(10, *, iostat=ierr) value
if (ierr == 0) then
! η ανάγνωση πέτυχε
else
! φτάσαμε στο τέλος ή προέκυψε σφάλμα
end if
```

Στο παράδειγμα, `ierr==0` σημαίνει ότι η τιμή διαβάστηκε κανονικά. Όταν το πρόγραμμα προσπαθήσει να διαβάσει μετά την τελευταία εγγραφή, το `ierr` παίρνει τιμή διαφορετική από 0. Τότε βγαίνουμε από τον βρόχο με `EXIT`.

Τι σημαίνουν τα IOSTAT και IERR

Το `iostat` είναι λέξη-κλειδί της Fortran και προέρχεται από το Input/Output Status, δηλαδή «κατάσταση εισόδου/εξόδου». Ζητά από τη `READ` να τοποθετήσει τον κωδικό επιτυχίας ή αποτυχίας σε μια ακέραια μεταβλητή. Το `ierr` δεν είναι δεσμευμένη λέξη της Fortran: είναι απλώς το όνομα μεταβλητής που επιλέξαμε. Συνηθίζεται επειδή θυμίζει I/O error ή integer error. Θα μπορούσε να ονομάζεται `status`, `io_code` ή οποιοδήποτε έγκυρο όνομα, αρκεί να είναι `INTEGER`.

Μικρό παράδειγμα - τι συμβαίνει σε κάθε READ

Αν το `data.txt` περιέχει τις τρεις γραμμές 12.5, 8.0 και -3.2, η εντολή `read(10,*,iostat=ierr) value` λειτουργεί ως εξής:

Προσπάθεια	Επόμενη εγγραφή του data.txt	Τι αποθηκεύεται στο value	Τιμή του ierr
1η READ	12.5	value = 12.5	0
2η READ	8.0	value = 8.0	0
3η READ	-3.2	value = -3.2	0
4η READ	Τέλος αρχείου - δεν υπάρχει νέα γραμμή	Δεν χρησιμοποιούμε το value	Διαφορετική από 0

Δεν χρειάζεται να απομνημονεύσεις ποιος ακριβώς μη μηδενικός αριθμός θα εμφανιστεί: αυτό μπορεί να εξαρτάται από τον compiler και από το είδος του προβλήματος. Για τον βρόχο μας αρκεί ο έλεγχος `ierr == 0` ή `ierr /= 0`. Στην 4η `READ` δεν εκτυπώνουμε ούτε αποθηκεύουμε το παλιό `value` - κάνουμε αμέσως `EXIT`.

6.7 Διάβασμα αρχείου αγνώστου πλήθους δεδομένων

Αυτή είναι η βασική τεχνική του Εργαστηρίου 9. Το παρακάτω πρόγραμμα διαβάζει έναν αριθμό από κάθε γραμμή και τον εμφανίζει. Δεν χρειάζεται να γνωρίζουμε από πριν το πλήθος των γραμμών.

```
program read_unknown
implicit none
integer :: ierr
real :: value

open(unit=10, file='newdata.dat', status='old', action='read')

do
read(10, *, iostat=ierr) value
if (ierr == 0) then
print *, value
else
exit
end if
end do

close(10)
end program read_unknown
```

Εξήγηση γραμμή-γραμμή: το do χωρίς όρια δημιουργεί βρόχο που θα συνεχίσει μέχρι να δώσουμε EXIT. Κάθε read προσπαθεί να πάρει την επόμενη τιμή. Αν πετύχει, ierr είναι 0 και εκτελείται το print. Αν δεν πετύχει επειδή τελείωσαν οι εγγραφές, η else εκτελεί exit. Μετά το end do, κλείνουμε το αρχείο.

Προσοχή: δεν χρησιμοποιούμε το value όταν η READ απέτυχε. Πρώτα ελέγχουμε το ierr και μόνο όταν είναι 0 επεξεργαζόμαστε την τιμή που μόλις διαβάστηκε.

6.8 Πολλά πεδία σε κάθε εγγραφή

Μια γραμμή μπορεί να έχει περισσότερες από μία τιμές. Για παράδειγμα, στο weatherdata.txt του εργαστηρίου κάθε εγγραφή περιέχει σχετική υγρασία και θερμοκρασία. Διαβάζουμε και τις δύο στην ίδια READ, με την ίδια σειρά που υπάρχουν στο αρχείο.

```
real :: humidity, temperature
...
read(10, *, iostat=ierr) humidity, temperature
```

Αν μια γραμμή του αρχείου είναι 55.0 23.4, τότε το humidity παίρνει 55.0 και το temperature παίρνει 23.4. Η σειρά είναι σημαντική: αν αντιστρέψουμε τις μεταβλητές, θα αντιστρέψουμε και το νόημα των δεδομένων.

6.9 REWIND και ανάγνωση σε δύο περάσματα

Συχνά ένα αρχείο έχει άγνωστο πλήθος γραμμών, αλλά θέλουμε τελικά να αποθηκεύσουμε όλες τις τιμές σε συστοιχίες. Για να γίνει αυτό καθαρά, το διαβάζουμε δύο φορές: την πρώτη φορά μόνο μετράμε τις εγγραφές. Μετά κάνουμε `rewind(10)`, που μεταφέρει τον δείκτη ξανά στην αρχή του ήδη ανοιχτού αρχείου. Στη δεύτερη ανάγνωση μπορούμε να δεσμεύσουμε συστοιχίες σωστού μεγέθους και να τις γεμίσουμε.

```
n = 0
do
  read(10, *, iostat=ierr) humidity, temperature
  if (ierr /= 0) exit
  n = n + 1
end do

rewind(10)
allocate(h(n), t(n))

do i = 1, n
  read(10, *) h(i), t(i)
end do
```

Το `n` αρχίζει από 0 και αυξάνεται μόνο για επιτυχημένες εγγραφές. Μετά τον πρώτο βρόχο, το αρχείο βρίσκεται στο τέλος του. Η `rewind(10)` δεν κλείνει το αρχείο και δεν σβήνει τίποτα: απλώς τοποθετεί την επόμενη ανάγνωση ξανά στην πρώτη γραμμή. Μόνο τότε η δεύτερη ανάγνωση θα ξαναβρεί τα δεδομένα.

Οι `h` και `t` είναι allocatable, επειδή το μέγεθός τους `n` δεν ήταν γνωστό όταν δηλώθηκαν. Με `allocate(h(n),t(n))` δεσμεύουμε ακριβώς όσο χώρο χρειάζεται. Μετά τη χρήση, γράφουμε `deallocate(h,t)`.

Πού δηλώνονται τα `h` και `t`;

Στο μικρό απόσπασμα πιο πάνω οι δηλώσεις έχουν παραλειφθεί για να φανεί μόνο η τεχνική των δύο περασμάτων. Πριν από το `OPEN`, στο δηλωτικό μέρος του κυρίως προγράμματος, θα υπήρχαν για παράδειγμα:

```
integer :: i, n, ierr
real :: humidity, temperature
real, allocatable :: h(:), t(:)
```

Το `h(:)` και το `t(:)` δηλώνουν πίνακες των οποίων το μήκος δεν έχει αποφασιστεί ακόμη. Η λέξη `allocatable` επιτρέπει να αποφασίσουμε το μήκος αργότερα, όταν το πρώτο πέρασμα βρει το `n`. Οι ίδιες δηλώσεις εμφανίζονται ολοκληρωμένες αμέσως παρακάτω, στο παράδειγμα `weather_summary`.

Θα μπορούσε να γίνει χωρίς ALLOCATE;

Ναι, αλλά μόνο αν αποφασίσουμε από την αρχή ένα σταθερό μέγιστο πλήθος θέσεων. Τότε ο πίνακας δημιουργείται ήδη όταν ξεκινά το πρόγραμμα και δεν χρειαζόμαστε allocate:

```
integer, parameter :: max_n = 1000
real :: h(max_n), t(max_n)
```

Με αυτή τη λύση έχουμε πάντα 1000 θέσεις, ακόμη και αν το αρχείο έχει μόνο 20 γραμμές. Επιπλέον, πρέπει να ελέγχουμε ότι το n δεν ξεπερνά το max_n. Εφόσον στο εργαστήριο το πλήθος είναι άγνωστο, η allocatable λύση δεσμεύει ακριβώς όσο χώρο χρειάζεται και είναι η πιο σωστή επιλογή.

Γιατί δεν γράφουμε απλώς real :: h(:);

Στο κυρίως πρόγραμμα, η γραμμή real :: h(:) δεν είναι έγκυρη. Η Fortran βλέπει πίνακα άγνωστου μεγέθους, αλλά δεν της έχουμε πει ούτε ένα σταθερό μέγεθος ούτε ότι θα δεσμεύσουμε χώρο αργότερα. Για δυναμικό πίνακα στο κυρίως πρόγραμμα γράφουμε υποχρεωτικά real, allocatable :: h(:) και, αφού βρούμε το n, allocate(h(n)).

```
real, allocatable :: h(:)
! ... μετά την πρώτη DO, όταν το n είναι γνωστό
allocate(h(n))
```

Πότε βλέπουμε h(:) χωρίς ALLOCATABLE;

Η μορφή h(:) χωρίς allocatable εμφανίζεται συνήθως ως όρισμα μιας SUBROUTINE ή FUNCTION. Εκεί ο πίνακας έχει ήδη δημιουργηθεί από το κυρίως πρόγραμμα και η διαδικασία απλώς δέχεται οποιοδήποτε μήκος πίνακα. Δεν κάνει η ίδια allocate για το h.

```
program main
real :: values(3)
values = [1.0, 2.0, 3.0]
call show_sum(values)
contains
subroutine show_sum(h)
real, intent(in) :: h(:)
print *, sum(h)
end subroutine show_sum
end program main
```

Στο παράδειγμα, το values(3) δημιουργείται στο κυρίως πρόγραμμα με σταθερό μέγεθος. Όταν καλείται η show_sum(values), το όρισμα h(:) βλέπει αυτόν τον ήδη υπάρχοντα πίνακα και η size(h) είναι 3. Άρα: h(:) ως όρισμα διαδικασίας δεν απαιτεί allocate, ενώ h(:) ως πίνακας που θέλουμε να δημιουργήσουμε στο κυρίως πρόγραμμα χρειάζεται allocatable και allocate.

6.10 Ολοκληρωμένο παράδειγμα: weatherdata.txt

Το επόμενο πρόγραμμα υλοποιεί τον βασικό κορμό της εργασίας: μετρά τις εγγραφές, αποθηκεύει την υγρασία και τη θερμοκρασία σε allocatable συστοιχίες και υπολογίζει τους μέσους όρους.

```
program weather_summary
implicit none
integer :: i, n, ierr
real :: humidity, temperature
real, allocatable :: h(:), t(:)

open(unit=10, file='weatherdata.txt', status='old', action='read')

n = 0
do
read(10, *, iostat=ierr) humidity, temperature
if (ierr /= 0) exit
n = n + 1
end do

rewind(10)
allocate(h(n), t(n))
do i = 1, n
read(10, *) h(i), t(i)
end do
close(10)

print '(a,f8.2)', 'Μέση υγρασία: ', sum(h)/real(n)
print '(a,f8.2)', 'Μέση θερμοκρασία: ', sum(t)/real(n)
deallocate(h, t)
end program weather_summary
```

6.11 Τι εμφανίζει και γιατί

Το `weather_summary` δεν εμφανίζει όλες τις αρχικές εγγραφές: εμφανίζει δύο γραμμές, μία για τη μέση υγρασία και μία για τη μέση θερμοκρασία. Οι πραγματικές τιμές εξαρτώνται από το περιεχόμενο του `weatherdata.txt`.

Στις δύο `PRINT`, η `sum(h)` προσθέτει όλες τις υγρασίες και η `sum(t)` όλες τις θερμοκρασίες. Η `real(n)` μετατρέπει το πλήθος των εγγραφών σε πραγματικό αριθμό, ώστε η διαίρεση να είναι πραγματική. Το `f8.2` εξασφαλίζει ότι η τιμή εμφανίζεται με δύο δεκαδικά ψηφία.

Για τις συνθήκες άνεσης του εργαστηρίου, μπορούμε να ελέγξουμε κάθε εγγραφή με `IF`. Η άνετη υγρασία είναι από 40 έως 60% και η άνετη θερμοκρασία από 22 έως 25.5 βαθμούς Κελσίου.

```
if (h(i) >= 40.0 .and. h(i) <= 60.0 .and. &  
t(i) >= 22.0 .and. t(i) <= 25.5) then  
  print '(2(f8.2,1x))', h(i), t(i)  
end if
```

Η συνθήκη είναι αληθής μόνο όταν και τα τέσσερα κριτήρια ισχύουν. Το `1X` είναι περιγραφέας μορφοποίησης για κενό: ο αριθμός πριν από το `X` δηλώνει πόσους κενούς χαρακτήρες θέλουμε. Άρα `1X` σημαίνει «άφησε ένα κενό». Το `2` πριν από την παρένθεση σημαίνει «επανάλαβε το `f8.2,1x` δύο φορές», δηλαδή ουσιαστικά `f8.2,1x,f8.2,1x`. Έτσι το `h(i)` και το `t(i)` εμφανίζονται με ένα κενό ανάμεσά τους. Το τελευταίο `1X` απλώς αφήνει ένα κενό στο τέλος της γραμμής και δεν αλλάζει ορατά το αποτέλεσμα.

6.12 Από εκφώνηση σε σχέδιο προγράμματος

Για πρόβλημα αρχείου αγνώστου πλήθους, ακολούθησε σταθερά αυτό το σχέδιο: (1) δήλωσε τις προσωρινές μεταβλητές, τον `ierr`, το `n` και τις `allocatable` συστοιχίες. (2) άνοιξε το αρχείο με `status='old'`. (3) κάνε πρώτο `DO` με `iostat` μόνο για καταμέτρηση. (4) κάνε `rewind`. (5) `allocate` με μέγεθος `n`. (6) διάβασε ξανά τα δεδομένα στον πίνακα. (7) υπολόγισε, έλεγξε και εκτύπωσε τα αποτελέσματα. (8) `close` και `deallocate`.

Μικρή υπενθύμιση πριν την εκτέλεση

Πριν πατήσεις `Run`, επιβεβαίωσε τρία πράγματα: το αρχείο υπάρχει στον φάκελο του `project`, το όνομα στο `file='...'` είναι ακριβώς ίδιο και η σειρά των μεταβλητών της `READ` ταιριάζει με τη σειρά των στηλών του αρχείου. Αν ένα από αυτά είναι λάθος, ο σωστός υπολογισμός που ακολουθεί δεν έχει νόημα, γιατί το πρόγραμμα δεν θα διαβάσει τα δεδομένα που περιμένεις.

6.13 Συχνά λάθη

- Λάθος όνομα ή λάθος θέση αρχείου. Έλεγξε ότι το αρχείο βρίσκεται στον φάκελο του project και ότι το όνομα στο file='...' είναι ίδιο.
- Χρήση status='old' για αρχείο που δεν υπάρχει. Το πρόγραμμα δεν μπορεί να το ανοίξει.
- Ανάγνωση με λάθος σειρά μεταβλητών σε σχέση με τις στήλες του αρχείου.
- Επεξεργασία της τιμής πριν ελέγξουμε ότι ierr==0.
- Ξεχασμένο rewind πριν από τη δεύτερη ανάγνωση. Τότε το αρχείο παραμένει στο τέλος και δεν διαβάζεται τίποτα.
- Χρήση h(i) ή t(i) πριν από allocate.
- Παράλειψη close ή deallocate στο τέλος.
- Ακέραια διαίρεση στον μέσο όρο. Για πραγματικό αποτέλεσμα γράφουμε real(n).

6.14 Ασκήσεις εφαρμογής

1. Δημιούργησε αρχείο numbers.txt με πέντε πραγματικούς αριθμούς, έναν σε κάθε γραμμή. Γράψε πρόγραμμα που το ανοίγει, διαβάζει τις τιμές σε πίνακα και εμφανίζει το άθροισμά τους.
2. Γράψε πρόγραμμα που ανοίγει το αρχείο input.txt, το οποίο περιέχει άγνωστο πλήθος πραγματικών αριθμών, και εμφανίζει κάθε αριθμό. Χρησιμοποίησε do, iostat και exit.
3. Για το αρχείο της Άσκησης 2, μέτρησε το πλήθος των αριθμών με πρώτο πέρασμα, κάνε rewind και αποθήκευσέ τους σε allocatable πίνακα.
4. Γράψε πρόγραμμα που διαβάζει από το measurements.txt ζεύγη τιμών x και y. Υπολόγισε και εμφάνισε τους μέσους όρους των δύο στηλών.
5. Γράψε πρόγραμμα που δημιουργεί το report.txt με status='replace' και γράφει σε αυτό έναν τίτλο, το πλήθος των δεδομένων και τον μέσο όρο τους με δύο δεκαδικά.
6. Με βάση το weatherdata.txt, εκτύπωσε τις εγγραφές με υγρασία 40-60% και θερμοκρασία 22-25.5 βαθμούς. Έπειτα υπολόγισε το ποσοστό των εγγραφών που είναι άνετες.
7. Τροποποίησε την Άσκηση 6 ώστε να γράφει τις άνετες εγγραφές σε νέο αρχείο comfortable_days.txt με δύο δεκαδικά ψηφία.

Στην ενότητα αυτή δίνονται μόνο εκφωνήσεις. Οι λύσεις θα δοθούν σε ξεχωριστό τεύχος.

6.15 Checklist Κεφαλαίου 6

- Μπορώ να εξηγήσω τι είναι unit, record και field.
- Μπορώ να ανοίξω υπάρχον αρχείο με OPEN και status='old'.
- Μπορώ να δημιουργήσω αρχείο αποτελεσμάτων με status='replace'.
- Μπορώ να διαβάσω τιμές με READ και να γράψω αποτελέσματα με WRITE.
- Ξέρω γιατί γράφουμε CLOSE.
- Μπορώ να χρησιμοποιήσω iostat για άγνωστο πλήθος εγγραφών.
- Ξέρω τι κάνει η REWIND και πότε τη χρειάζομαι.
- Μπορώ να κάνω allocate - δεύτερη ανάγνωση - deallocate για δεδομένα αγνώστου πλήθους.
- Μπορώ να συνδυάσω arrays, DO και IF για να αναλύσω δεδομένα από αρχείο.